ELI — Security

ELI v2.5.31

November 2026

Contents

ELI Security Posture **1**
1. Shell command gate (`runtime/security.py` `SecurityManager`) . . . 1
2. Filesystem & app gates (`SecurityManager`) . . . 2
3. Prompt-injection guard (`kernel/engine.py`) . . . 2
4. SQL identifier validation (`memory/memory.py`) . . . 2
5. Custom-agent trust chain (`cognition/agent_trust.py`) . . . 2
6. Code generation & self-modification — three paths . . . 2
Honest assessment . . . 3
RUN_CMD terminal is real; mock leak fenced . . . 4
Web app & multi-user security . . . 4
Stable phone token, admin-gated model switch . . . 5
Token moved to the URL fragment; test reds cleared . . . 6
Community plugin marketplace: consent, verification, scanning . . . 6

ELI Security Posture

> **Updated for v2.4.84.** Fail-closed shell gate, path allow-roots, custom-agent SHA-256 trust registry, prompt-injection sanitisation at engine entry (S02).

ELI runs locally with real OS reach (shell, file, app control) plus a user-extensible agent/plugin system — so the security model matters. It is **fail-closed** by design. Spans `eli/runtime/security.py`, the executor gate, the engine input sanitiser, the memory SQL validator, and the agent trust registry.

1. Shell command gate (`runtime/security.py` `SecurityManager`)

- `is_command_allowed(cmd)`:
  - **ELI Full Control** on (`is_full_control()`) → allow all.
  - `ELI_ALLOWED_CMDS` contains * → allow all.
  - `ELI_ALLOWED_CMDS` unset **and** Full Control off → **blocked (fail-closed)**; logs a warning explaining how to opt in. (Always allows the project's own `.venv/bin/python`.)
- `safe_subprocess(cmd, timeout)` — validates `cmd[0]` against the allowlist before running; capture_output, timeout-bounded, never `shell=True`.
- **Defence in depth:** `executor_enhanced._shell_command_allowed_fallback` mirrors this exact logic, so if `SecurityManager` fails to import, the executor still fails **closed** (never fail-open). Explicit comment to that effect.

> **ELI Full Control has no environment variable — deliberately.** The single source of truth is the `full_control` setting flipped by the GUI toggle, read live via `core/full_control.py` `is_full_control()`; nothing in the process environment can turn it on, so no stray export can silently unlock the gates, and flipping the toggle off restores every gate at once without a

restart. It lifts four barriers together: network gating, self-code patching, autonomy approval, and this command gate.

2. Filesystem & app gates (`SecurityManager`)

- `is_path_allowed(path)` — must resolve within allow-roots (`project_root` + `$HOME` + `ELI_ALLOW_ROOTS`). Uses `Path.resolve()` + `relative_to`, so `..` traversal can't escape.
- `is_app_allowed(app)` — explicit `ELI_ALLOWED_APPS`, else a curated default-safe set (settings, file manager, editor, calculator, terminal, browser, mail). ELI Full Control bypasses.

3. Prompt-injection guard (`kernel/engine.py`)

`_ELI_INJECTION_PATTERNS` (regex) + `_eli_sanitize_user_input`: - Matches classic jailbreak/role-override prefixes — `### system:`, "ignore/disregard/forget (all) previous instructions", "you are now a dan/jailbreak/unrestricted/free" — and replaces the matched span with `[filtered]` (keeps the rest of the message). - Strips control characters. - Runs before the input reaches the LLM, so injected role-overrides are neutralised but legitimate content survives.

4. SQL identifier validation (`memory/memory.py`)

`_IDENTIFIER_RE = ^[A-Za-z_][A-Za-z0-9_]*$` + `_validate_identifier(name)` — called before **any** f-string interpolation of a table/column name into SQL (`_validate_identifier(table, "table")`, column DDL first-token check). Raises `ValueError` on anything non-conforming. Closes the one place dynamic SQL is unavoidable (parameterised values are used everywhere else).

5. Custom-agent trust chain (`cognition/agent_trust.py`)

Custom agent code is **SHA-256 gated** and fail-closed: unknown, modified or revoked files are not loaded. `ELI_TRUST_ALL_AGENTS=1` bypasses for dev only.

Rebuilt in 2.3.7. The previous registry stored `{basename: sha256}` in `config/trusted_agents.json`, which had three defects that look fine until they don't:

- **Keyed on the basename.** Two files called `helper.py` in different directories shared one entry, so approving either silently authorised the other. Identity is now the **resolved absolute path**; the basename is kept for display only.
- **No provenance.** A bare hash cannot answer *who approved this, when, and what did it look like then* — the questions asked after something goes wrong. A grant now records the timestamp, the approver, the file size, the static-analysis verdict at approval, and the spec hash the code was paired with.
- **Nothing looked at the code.** The gate proved a file was unchanged since approval; it never asked whether approving it was reasonable. `grant()` now runs the same engines as the plugin marketplace and **refuses outright** on a malicious verdict (overridable with `force=True`, which is itself recorded).

Revocation is real: a revoked entry is *kept and marked revoked* rather than deleted, so re-adding the same file does not silently re-trust it. A v1 registry is migrated rather than discarded, with migrated entries flagged `legacy` so it stays visible that they predate these checks.

Most custom agents need none of this. An `AgentSpec` is data — objective, system prompt, triggers, success criteria — so a spec-driven agent executes no arbitrary code and never touches the trust gate. See `orchestration_and_agents.md`.

6. Code generation & self-modification — three paths

ELI has **three** distinct codegen paths, with escalating capability and matching safeguards:

1. **Pre-vetted canned library** (`runtime/generated_script_guard.py`, 1.1k LOC) — ~5 hand-written, reviewed scripts (`_write_gpu_memory_watch_script`, `_write_type_ia_redshift_script`, `_write_ton_618_mass_density_script`, ...) that ELI *writes to disk* on request. No LLM code is executed; safe but fixed to the curated set (several physics-domain).
2. **`GENERATE_SCRIPT`** (`execution/executor_enhanced.py`) — *real* LLM codegen. Detects language/intent, prompts for frontier-quality code, then gates it:
   - `_verify_python_module_apis` — imports the referenced libraries and confirms `module.attr` references actually exist (catches hallucinated APIs).
   - `_quality_reject_reason` — rejects refusals, stubs, too-short, no-real-compute, missing-plot, `required=True`-without-default, SyntaxError.
   - **`_sandbox_run_python` (added)** — executes the candidate in a bounded, isolated subprocess (temp cwd, scrubbed env, `MPLBACKEND=Agg` so `plt.show()` can't block, wall-clock timeout `ELI_GENSCRIPT_RUN_TIMEOUT`=20s, generous `RLIMIT_CPU`; **no `RLIMIT_AS`** — it breaks numpy/scipy). A genuine unhandled traceback is fed back as repair feedback into a **3-attempt regenerate loop**; timeouts / signal kills / missing-optional-deps are tolerated (never reject legit heavy scientific scripts). Disable with `ELI_GENSCRIPT_VERIFY_RUN=0`.
3. **Self-patching** (`runtime/self_improvement.py SelfImprovementEngine`) — ELI modifies its *own* source for recurring failures. Gated behind `auto_patch_enabled` (**default off**), ≤2 patches/cycle. Safeguards: verbatim-old-match → `ast.parse` → timestamped backup (+ canonical `.eli_bak`) → write → `py_compile` → **differential import smoke-test (added)**: the patched module is imported in an isolated subprocess and **auto-reverted** if it no longer loads, but only when the module imported cleanly *before* the patch (so missing optional deps don't cause false reverts). Project-root-confined, `.py`-only, logged to the `code_patches` table.

   Code-health flag: `generated_script_guard` uses the same stacked-override pattern as the grounding gate — two `install()` definitions chained via `_ELI_SQLITE_SCRIPT_PREVIOUS_INSTALL`. Works, but layered rather than edited.

   Possible next step (not done): the self-patch loop verifies the module still *imports*; it does not yet re-run the original failing input to confirm the patch actually *fixes* the failure. A behavioural/test re-run would close that last gap.

Honest assessment

- **Strong:** the gates are real and **fail-closed** — shell blocked by default, fallback mirror prevents fail-open, path traversal contained, identifiers validated, agents hash-trusted, injection prefixes stripped, and "generated" code is actually pre-reviewed. This is well above the norm for local-AI projects, which routinely ship wide-open shell access.
- **Weak / watch:**
  1. The injection guard is **pattern-based** — it catches known prefixes, not novel/obfuscated injections (no model-side defence). Reasonable for a single-user local tool, insufficient if ever multi-tenant.
  2. **ELI Full Control** is a single toggle that disables *all* gates at once — convenient, but a blunt instrument; there's no middle "allow file but not shell" tier beyond the per-axis env vars.
  3. ~~The default-safe app/command sets are Linux/GNOME-flavoured.~~ **RESOLVED** — the default-safe **app** set in `_is_default_safe_app` is cross-platform (Linux + macOS Finder/Safari/Terminal.app/System Settings + Windows Explorer/Notepad/cmd/PowerShell/ Control Panel). The **command** path is fail-closed (no OS-specific default set — blocked unless `ELI_ALLOWED_CMDS`/Full-Control), so nothing OS-specific to add there.
  4. Pre-vetted-scripts approach is safe but doesn't scale — genuinely dynamic capability generation would need a real sandbox (resource limits, seccomp, subprocess isolation), which doesn't exist yet.

RUN_CMD terminal is real; mock leak fenced

- **RUN_CMD uses a real `subprocess.run`** (`executor_enhanced.py:5350` — no shell, capture_output, timeout) behind the destructive-command security gate (`_BLOCKED_PATTERNS`: `rm -rf /`, `mkfs`, `dd of=/dev/`, `chmod 777 /`, fork bomb, shutdown/reboot). The gate is lifted only when **Full Control** is on. There is **no production mock path**.
- **The `<MagicMock …>` failure rows were test leakage, not runtime.** They came from `tests/test_shell_security_gate.py` patching `subprocess.run`; the executor's `(p.stdout or "") + (p.stderr or "")` concat then yields a MagicMock repr, which slipped into the live `agent.sqlite3` via a test-isolation gap. A guard in `SelfImprovementEngine.log_failure` now **drops any error/input carrying a Mock/MagicMock repr** at the write source, so a test isolation slip can no longer pollute real failures.

Web app & multi-user security

ELI now ships a self-hosted FastAPI web app (`api/server.py`). It is local-first and inherits all of the gates above (the model is the same local GGUF behind netguard; commands/files/apps stay fail-closed). The web surface adds:

7. Authenticated identity + RBAC (`eli/runtime/api_users.py`, `api/server.py`)

- **Three roles** — `admin` / `member` / `viewer` (read-only). Endpoints are dependency-gated (`require_admin` / `require_member` / `require_viewer` / `_require_token`). RBAC enforces once at least one user exists; before that, same-machine use is frictionless.
- **Fail-closed auth gate** — unauthenticated/under-privileged calls are rejected, not silently downgraded. The token store holds **hashes**, never raw tokens.

8. Tamper-evident, HMAC-keyed audit trail (`eli/runtime/evidence_ledger.py`)

- Every API action is recorded into a **hash-chained** ledger (who / what / outcome, **metadata only — never message content**). Any edited/deleted/reordered row is detected by `verify_chain()` and surfaced in the Audit tab + admin console.
- The chain is **HMAC-keyed** (`$ELI_AUDIT_HMAC_KEY`, else a `0600` key file beside the config), so the integrity check can't be forged by recomputing plain hashes.

9. Secret files born locked — TOCTOU closed (`eli/core/secure_io.py`)

- The old `write_text(...)` then `chmod(0o600)` pattern left a brief window where, on a typical umask, the file was born `0644` (world-readable) before the chmod landed — a real (if narrow) TOCTOU on a multi-user box, on exactly the files that matter most.
- **`secure_io.secure_write_text/bytes`** is now the single owner of secret writes: `tempfile.mkstemp` (born `0600`) → write → fsync → `os.replace` (atomic). The destination **never exists world-readable for any instant**, and readers see old-or-new, never partial.
- Routed through it: the **audit HMAC key**, the **API token store**, **settings** (may hold broker passwords/tokens), and the **agent-trust registry** (`trusted_agents.json`, the integrity anchor for §5 — hashes not secrets, but written `0600`/atomic so it can't be tampered or half-written).

10. Sandboxed research ingest

- Corpus ingest for the research workspace runs sandboxed (path-scoped), so a shared/ collaborative corpus can't reach outside its workspace.

11. Monitored Internet toggle + egress oversight (`eli/core/netguard.py` + `api/server.py`)

- ELI is **offline-by-default** and hard-gated at the **socket boundary** — the process-wide failsafe raises `OfflineError` on any non-loopback connect while the toggle is off, even from code that forgot the helpers (fail-closed; loopback/LAN-registered services allowed).
- The guard covers **all four** ways an outbound connection actually happens, cross-platform: `socket.socket.connect` (raises) and `socket.create_connection` (raises) — sync + the selector asyncio loop / urllib / requests; `socket.socket.connect_ex` — the **non-raising sibling** used by probes and health-checks, which is made to **return `ENETUNREACH`** when blocked (no socket is opened) so it can't silently bypass the block or the recorder; and a **best-effort patch of the Windows `ProactorEventLoop`** (`IocpProactor.connect`, overlapped `ConnectEx`), which never touches `socket.socket.connect`. The allowed path of each records egress identically (below); the Proactor patch is a no-op on non-Windows.
- The owner can deliberately **enable internet**: from the desktop GUI (Net toggle) or the web dashboard (`GET /v1/net` token-gated read + `POST /v1/net` **admin-only**), persisted via `network_enabled`. **Both flips are written to the audit ledger** (`net_toggle`, warning-severity on enable) — the web *and* the desktop toggle.
- **Egress is genuinely monitored, not a blind hole.** While network is on, the same socket choke-point that enforces offline mode also **records every allowed non-loopback connection** (host:port + timestamp): into an **in-memory ring** (live tail; `GET /v1/net/egress` + the Overview widget) and, throttled to one row per host:port per window (`ELI_EGRESS_LOG_WINDOW`, default 300s), into the **tamper-evident audit ledger** as `net_egress` events. The ledger write is best-effort and runs on a background thread, so monitoring never delays or breaks a connection. Disable the ledger leg (keep the live ring) with `ELI_EGRESS_LEDGER=0`.
- The toggle changes *policy*; the socket failsafe still governs every actual connection, and every connection it allows is now on the record.
- **No library reports to its maker** (2.5.14). Usage telemetry built into the libraries ELI carries is switched off when the `eli` package loads (`ORT_DISABLE_TELEMETRY`, `HF_HUB_DISABLE_TELEMETRY`, `GRADIO_ANALYTICS_ENABLED`, `DO_NOT_TRACK`, `OTEL_SDK_DISABLED`, `ANONYMIZED_TELEMETRY`, `SCARF_NO_ANALYTICS`). onnxruntime's sends from native code, which the socket guard cannot see, so its switch is the only thing that stops it. Hosts that exist only to collect telemetry or crash reports (`netguard.REPORTING_HOSTS`) never resolve, even with the Net switch on (`tests/test_netguard_socket_guard.py`).
- Rationale: internet is "the final frontier to make ELI's world bigger" — available, but monitored (per-connection) and under the user's control, not a permanent lockout.

Honest assessment — web tier

- **Strong:** fail-closed auth, role separation, a tamper-evident HMAC'd audit trail, born-locked secret files, and a monitored (not absent) network path. For a self-hosted local AI this is well above the norm.
- **Watch:** RBAC is token-based and single-tenant-shaped; the injection guard (§3) is still pattern-based; ELI Full Control remains a blunt all-gates-off switch. None new, but they matter more once the web app is exposed beyond the owner's machine — keep it bound to trusted networks.

Stable phone token, admin-gated model switch

Two web-tier changes this cycle, both verified against the code:

- **Stable LAN token + rotate.** The phone's bearer token now **persists** across server restarts (`api/api_token.py`: env → a `0600` file under the config dir → generate-and-save), so a paired phone is no longer stranded every time the server bounces. A **rotate** button issues a fresh token and invalidates the old one in one tap — the manual override for a lost/compromised phone. `/health` stays tokenless by design.
- **Model switching is admin-gated and allowlist-bound.** The dashboard's model dropdown posts to `POST /v1/model`, which requires an **admin** principal and only accepts a path already in the

installed-models list (`GET /v1/models/installed`). A dropdown of real files can't strand the runtime on a bad path, and a non-admin can't switch the model. (The list endpoint was moved off `/v1/models` to `/v1/models/installed` so it no longer collides with the OpenAI-compatible route.)

Neither weakens the existing posture — the socket failsafe, fail-closed command gate, and audit ledger all still apply.

Token moved to the URL fragment; test reds cleared

- **Token in the URL fragment, never the query string.** Every emitter of the phone link now builds `…/#token=…` instead of `…/?token=…`: the server's printed URLs *and* the desktop **Web Server panel** (`eli/gui/eli_pro_audio_gui_v2_0.py:8332/:8391`) *and* both launchers (`scripts/eli_serve.sh`, `scripts/eli_serve.ps1`). A query string reaches the server — it lands in **uvicorn's access log** before any client-side strip can run — whereas a **fragment is never sent to the server**, so the token can't leak into the log. The page reads it from `location.hash`, stores it, and clears the address bar; it still accepts `?token=` for old links, so the transition breaks nothing. (An earlier commit fixed only the web tab + printed URL, leaving the GUI/launcher paths — most real usage — still leaking; this closes them.)
- **All 5 standing test reds cleared.** Deprecated `smart_home` plugin removed; 113 silent `except: pass` swallows made observable (987→874, ceiling 950→900); stale blueprint references fixed. Not a security change per se, but the suite is now fully green.

Community plugin marketplace: consent, verification, scanning

ELI ships a marketplace **client**; the marketplace itself belongs to the community. Nobody curates it, so the usual first line of defence — "the store checked it" — does not exist, and ELI must never imply otherwise. Everything below therefore comes from the artifact itself, and every screen says exactly what was checked and what was not.

12. Runtime permissions (`eli/plugins/permissions.py`)

A plugin was previously ordinary Python `exec_module`'d inside ELI's process: whatever the interpreter could do, the plugin could do — read every conversation, reach the network, drive the mouse — with nothing declared and nobody asked.

Plugins now **declare** capabilities in their manifest and the operator is asked, in plain language, before any of them is used. Thirteen capabilities (`network`, `filesystem_read/write`, `process_exec`, `os_control`, `screen_capture`, `camera`, `microphone`, `memory_read/write`, `model_access`, `clipboard`, `notifications`, `audio_playback`), each with a risk level and a sentence on why it is risky, phrased as what it can do *to you*.

Four answers — **Allow once · Always allow · Not now · Never allow**. Only the two permanent ones persist; "once" is session-scoped and dies with the process. "Never" is remembered and the plugin is **never asked again**, which is what stops nagging.

Three rules hold everywhere:

- **Fail closed.** No prompt handler registered — headless, API server, a scheduled task at 3am — means DENY, never a silent allow. A plugin cannot escalate by running where nobody is watching. A crashed or unresponsive dialog also denies.
- **Nothing is granted by installing.** Install consent and use consent are separate acts; Android learned this the hard way with install-time permissions.
- **Every decision is audited** to a JSONL ledger the operator can read back, and any grant can be revoked from Settings ▸ Marketplace ▸ Permissions.

An unrecognised capability is reported as **critical**, not ignored — a build that cannot explain a permission cannot enforce it either.

13. Manifest vs. code (`eli/plugins/manifest.py`)

The manifest declares; the code is statically checked against it, and **undeclared capability use is a refusal, not a warning**. A plugin that quietly imports `subprocess` while declaring nothing is exactly the case this stops.

Some constructs cannot be declared away and are refused outright, because they defeat every static check above: `eval`, `exec`, `compile`, `__import__`, `importlib.import_module`, `marshal.loads`, `pickle.loads`, `ctypes.CDLL`.

The scan is deliberately conservative — it over-reports rather than under-reports. A false "you must declare filesystem_write" costs a publisher one manifest line; a false negative costs an operator their files.

14. Integrity and publisher identity (`eli/plugins/integrity.py`)

Two narrow promises, both verifiable on the operator's own machine:

- **You got what the listing described.** The listing carries a sha256; the download is hashed *before* it is written to disk. A mismatch is a hard refusal — this is what stops a compromised mirror, a man-in-the-middle on a plain-http source, or a publisher swapping the file under an unchanged listing.
- **It came from a publisher you chose to trust.** Optional ed25519 signatures verified against keys the **operator** added. Not a key the ELI author holds — there is no central authority here by design. There is deliberately no trust-on-first-use shortcut: TOFU would make the first plugin from anyone trusted forever, which is the same as no check.

An unsigned community plugin is **not blocked** — it is reported as unverified, loudly. A *failed* signature is blocked. Hash checking is mandatory and dependency-free; signature checking needs `cryptography`, and where it is absent the plugin is reported as **unverifiable rather than verified**.

15. Malware scanning (`eli/plugins/security_scan.py`)

Eleven independent engines, run on the operator's machine as well as by whatever the registry did upstream — a scan you did not run yourself is a claim, not a result.

Engine	Looks for
`static_ast`	capability use, runtime code construction, dynamic imports
`obfuscation`	decode chains, char-code string building, large encoded literals
`ioc_patterns`	reverse shells, miners, hardcoded C2, paste/tunnel hosts, destructive commands
`credentials`	ssh keys, cloud credentials, browser stores, wallets, keychains, shell history
`persistence`	LD_PRELOAD, DYLD injection, process injection, cron, systemd, launchd, Run keys, shell rc files
`anti_analysis`	VM/debugger/sandbox detection — deliberate evasion
`entropy`	packed or encrypted blobs carried inside source
`dependencies`	typosquatted pip names, URL installs, installer-code warning
`hash_blocklist`	known-bad artifacts (local, community-updatable)
`clamav`	full antivirus, if `clamscan`/`clamdscan` is installed
`yara`	custom rules, if `yara-python` and a ruleset are present

Two rules the scoring obeys:

- **An engine that could not run never counts as a pass.** It is named as unavailable and the verdict says coverage was partial. A scanner that quietly downgrades to "clean" when ClamAV is missing is worse than no scanner.
- **Findings are evidence, not proof.** Each carries the line and the matched construct, so a publisher can argue with it and an operator can look.

Scoring is capped per category, so twenty matches of one pattern cannot outweigh three different kinds of evidence — breadth is the stronger signal. Any critical finding, or a score ≥ 45, or three high findings ⇒ **malicious**, and the install is refused.

16. The install path (`eli/plugins/marketplace.py`)

Deliberately slow, and ordered so nothing dangerous happens before the operator has seen the result:

1. validate the listing → 2. licence gate for paid plugins → 3. download **to memory** → 4. verify hash + signature → 5. static check against the manifest →
2. run every scanner → 7. show the operator everything → 8. only then write.

What is written arrives **switched off with no permissions granted**. The disabled flag is written straight to state rather than through `get_manager().disable()`, because constructing the manager runs `_auto_load()`, which would import and execute the freshly downloaded module before it could be marked off. Module-level code in a downloaded plugin must not run at install time.

`pip` dependencies need a **separate** approval: package installers run their own code in a child process and reach the network outside anything ELI can gate.

Registries are a **list, not a constant** — operators add the community sources they want and can remove any of them. Only the bundled offline index of ELI's own plugins ships. A single hardcoded URL would make the author the gatekeeper of a marketplace that is not theirs.

Downloaded plugins are written to the **user data dir**, not `eli/plugins/` inside the installation. The old `_plugins_dir()` returned the source tree, which is a read-only mount on a packaged build and also mixed a stranger's code in with ELI's own shipped plugins.

17. MCP servers — and the limit of netguard (`eli/plugins/mcp.py`)

Stated plainly because it matters: `netguard`'s socket guard is a monkeypatch of `socket.socket.connect` **inside ELI's own interpreter**. An MCP server is a separate OS process — usually `npx` or `uvx`, often Node or Go — whose network stack never touches Python's socket module. Demonstrated directly: with `ELI_OFFLINE=1` and the guard installed, the parent process is correctly refused with `OfflineError` while a child process, inheriting the same environment, connects straight out.

There is **no eBPF, seccomp, landlock, network-namespace or firewall integration** anywhere in ELI today. So:

- **In-process Python plugins** are covered by netguard at the socket boundary, plus the permission gate above. (A `ctypes` native call would bypass it, which is why `ctypes.CDLL` is refused outright by the manifest checker.)
- **MCP servers, `pip`, and any subprocess are not covered.** ELI's offline switch cannot stop them, and ELI cannot see what they send.

`mcp.network_caveat()` returns the sentence every MCP consent screen must carry, so no UI can imply a containment that does not exist. Real enforcement would need kernel-level egress control per platform — cgroup/eBPF or a network namespace on Linux, WFP on Windows, a network extension on macOS — routing the child through an ELI-owned proxy that netguard *can* govern. That work does not exist yet.

What the MCP layer *does* guarantee is that a configured server actually works: there is **one** config file ELI owns, installing runs a runtime preflight then a real `initialize` + `tools/list` handshake, and an

entry is written **only after a server has answered**. A missing runtime or a process that is not an MCP server is refused before the config is touched.

18. Fetching from sources the operator does not own (`eli/core/netguard.py`)

`guarded_urlopen` answers one question — is the network switch on? That is the right question for ELI's own calls and the wrong one for a URL that came from a community registry. urllib follows redirects silently and the socket guard permits loopback unconditionally, so a hostile listing could aim a marketplace fetch at ELI's own API server or a LAN device: server-side request forgery, with ELI as the confused deputy. Demonstrated at the time — a 302 to `http://127.0.0.1/...` was followed and the body handed back to the caller.

`safe_fetch` is what third-party content now goes through:

- **Scheme pinned** to http/https. `file:`, `ftp:` and `data:` can read local resources through the very same call.
- **Addresses filtered.** The host is resolved and refused if it lands on loopback, private, link-local (169.254.0.0/16 — cloud metadata), multicast or reserved space. `allow_private=True` exists for an operator running their own LAN registry and is never inferred from the URL, because that is what an attacker controls.
- **Every redirect hop re-validated.** Checking only the URL the caller passed is worth little; the interesting address is the last one, and the server picks it.
- **Body capped while reading** (32 MiB default, 8 MiB for a plugin), so a server omitting `Content-Length` cannot stream unbounded data into memory.
- **`Accept-Encoding: identity`** so a compressed response cannot expand past the cap, and an explicit verifying TLS context.

Stated rather than papered over: the host is resolved for validation and resolved again by the connection, so DNS rebinding between the two can still land on a private address. Closing that needs connecting to the validated IP while preserving SNI, which the stdlib opener does not expose cleanly.

A registry whose URL is private is recorded as `allow_private` **at add time**, with a warning. "Does not resolve" is a distinct error from "resolves somewhere private" — conflating them once made an unreachable public registry look like a deliberate LAN one and silently granted it local-network access.

19. Runtime capability enforcement (`eli/plugins/sandbox.py`)

Everything above gates a plugin *before* it runs. All of it is defeated by one fact: an enabled plugin is `exec_module`'d into ELI's own interpreter, so the permission API is **cooperative**. A plugin that passed the manifest check, the hash, the signature and eleven scanners can simply `import socket` and never call a gated helper.

`sys.addaudithook` fires below the Python API, on the operation itself — `socket.connect`, `open`, `subprocess.Popen`, `os.system`, `ctypes.dlopen` — and raising inside the hook aborts it. The hook attributes each event to a plugin by walking the stack for a frame inside a plugin package, then enforces that plugin's **declared** capabilities and the operator's consent.

Verified directly: a plugin declaring only `notifications` had raw `socket.connect`, raw `open()` and raw `subprocess.run` all blocked, while ELI's own file access was untouched.

Properties, stated precisely because "sandbox" oversells it:

- **It cannot be uninstalled.** CPython exposes no way to remove an audit hook.
- **It enforces below the API.** Importing `socket` directly does not help.
- **It is not a boundary against native code.** A plugin running arbitrary machine code through a compiled extension is outside anything in-process — which is why `ctypes.dlopen` needs `process_exec` and the manifest checker refuses `ctypes` outright.

- **It does not contain subprocesses.** A plugin granted `process_exec` spawns a real child and nothing here follows it. Same limit as netguard, which is why that capability is described to the operator as unlimited access.
- Loopback is exempt from `network`: ELI's own model server and local API live there.
- Plugins that shipped with ELI and have no manifest are treated as fully declared — breaking them to enforce a contract they never had would be a regression.

Installed before the first plugin is loaded, so no plugin's module-level code runs ahead of it. `ELI_PLUGIN_SANDBOX=0` disables it, which is a real downgrade and logs a warning.

20. One-click install — where consent is and is not implied

The quick path verifies, scans and installs on a single click, and stops for a full review when there is genuinely something to decide. Getting that line wrong is bad in both directions: block on ambient conditions and the quick path never fires, so the operator learns to click through the review dialog without reading it; block on nothing and the click is consent theatre.

Blocks (each a judgement only the operator can make): a non-clean scan; no checksum in the listing, so the file cannot be matched to what was described; requested permissions; PyPI dependencies; a price; a plain-http download URL.

Does not block, reported instead: ClamAV/YARA/blocklist not installed (partial coverage) and the plugin being unsigned — both the normal state of a community marketplace.

Even on the quick path the plugin arrives **switched off with no permissions**; enabling is a separate, explicit answer.

21. Hosting a registry (and why GitHub Pages fits)

A registry is one static file — `index.json` over HTTPS. That is the whole hosting requirement, which is deliberate: it keeps the marketplace something a community can run for free, and keeps ELI's author out of the position of owning, moderating, or being answerable for what strangers publish.

GitHub Pages happens to satisfy every constraint `safe_fetch` enforces — HTTPS with a valid certificate, a public address, no redirect games — so it passes without any special-casing. `tools/marketplace/registry_template/` is a working skeleton and `tools/marketplace/publish.py` generates listings.

The publisher-side failures worth designing against, because each one lands on *users* rather than the publisher:

Mistake	What every user sees
stale `sha256`	a hard integrity refusal — reads as "this publisher ships broken plugins"
no `sha256`	the one-click install stops on every machine
plain-http `source`	the one-click install stops on every machine
under-declared `permissions`	the client refuses the plugin outright

`publish.py` catches all four at publish time: it validates the manifest, checks the code against the declared permissions, runs the same eleven scanners the user's machine will run, computes the hash, and **refuses to emit a listing that would be rejected on the other end**. Signing is optional (`--new-key` / `--sign-key`); an unsigned plugin is reported as unverified rather than blocked, which is the honest default when nobody is curating.

Reviewing listings as pull requests gives a public record of who added what, with CI able to run `publish.py` over each change — without the registry operator hosting anything beyond a static file.